SIMATS ENGINEERING

Assessment Tool 2: Case Study

Course Name: Cloud Computing and Big Data Analytics
Course Code: CSA15

Course Outcome Covered

CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Bloom's Taxonomy: Precision (Level 3) | Question Count: 3 | Total Marks: 30

Code of Conduct

I **DEENADAYALAN P (Reg. No. 192521135)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P DEENDAYALAN

Case Study 1: Smart Retail Inventory Management System

1.1 Understanding of the Case

A retail chain wants a single inventory system that every store can see and update in real time. Point-of-sale (POS) terminals record every sale and stock movement, while managers use dashboards to check stock levels and sales trends. Both client types need to reach a common backend over the internet, that backend needs to remember years of transaction history without the retailer managing physical servers, and the connection between stores and the cloud must be locked down so that competitors or attackers cannot see stock or pricing data.

Core requirements identified:

- Real-time, two-way communication between distributed store devices and a central system
- Durable storage for product catalogues, transaction logs, and historical analytics datasets
- Automated demand forecasting rather than manual spreadsheet analysis
- Network-level protection (VPN, firewall) in addition to application-level access control

1.2 Proposed Cloud Architecture

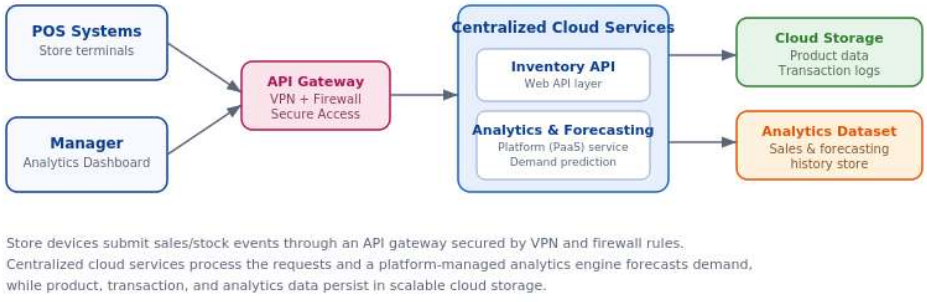


Fig 1: Cloud architecture for the Smart Retail Inventory Management System

1.3 Application of Cloud Concepts

Requirement / Component	Cloud Concept Applied	Justification

POS terminals & dashboards	Thin clients calling Web APIs	Keeps devices lightweight; all inventory logic and data live centrally, so every store sees the same stock figures instantly
API Gateway	Managed API Gateway (PaaS)	Provides one secure, throttled entry point instead of exposing backend services directly to store networks
Demand forecasting	Platform Service (PaaS) — managed ML/analytics	Retailer consumes a ready-made forecasting service instead of building and tuning models from scratch
Product data & transaction logs	Cloud Object/Block Storage + managed database	Elastic, pay-as-you-grow storage that scales automatically as more stores and years of data are added
Secure access	VPN + Firewall + API Gateway (IaaS + PaaS security)	Defence-in-depth: network-level VPN/firewall stops unauthorised traffic before it ever reaches the API layer

1.4 Analysis and Trade-offs

Centralising inventory in the cloud trades a small amount of network latency (every POS action now depends on connectivity) for a large gain in consistency: stock counts can never drift apart between stores the way they would with local databases synced overnight. Choosing a managed platform service for forecasting instead of a self-hosted model removes the operational burden of retraining and scaling machine-learning infrastructure, at the cost of some flexibility to customise the algorithm. Placing the API gateway in front of all services adds a small amount of processing overhead per request, but it is a worthwhile trade because it centralises authentication, rate-limiting, and logging in one place rather than repeating that logic in every backend service.

1.5 Solution Design Summary

The recommended design uses store-side POS/dashboard clients that never talk to storage directly. Every request passes through an API gateway protected by VPN and firewall rules, is handled by an inventory API and a managed analytics/forecasting service, and is persisted in cloud storage that separates fast-changing transactional data from larger historical analytics datasets. This keeps the store-facing footprint minimal while giving the retailer a single, consistent, and scalable view of inventory across all locations.

Case Study 2: Cloud-Based Document Collaboration System

2.1 Understanding of the Case

The enterprise needs a Google-Docs-like tool: many users, possibly on different continents, open the same document at the same time, see each other's edits almost instantly, and expect the same file to look identical whether they open it from a laptop browser or a phone. The system must therefore treat the browser as the only client software required, keep a full edit history rather than one overwritten

file, keep the experience fast regardless of the user's location, and control precisely who can view, comment on, or edit each document.

Core requirements identified:

- Real-time multi-user editing and sharing, accessible purely through a browser
- Distributed storage with version control so no edit is ever silently lost
- Global low-latency access and high availability across regions
- Strong identity management, encryption, and per-document access permissions

2.2 Proposed Cloud Architecture

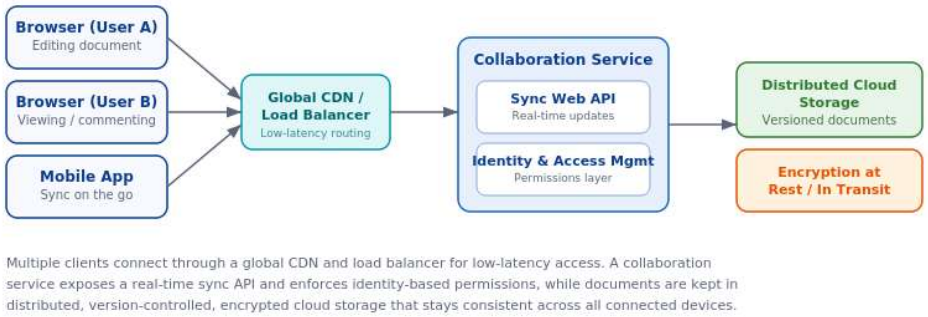


Fig 2: Cloud architecture for the Document Collaboration System

2.3 Application of Cloud Concepts

Requirement / Component	Cloud Concept Applied	Justification
Browser clients (multi-device)	SaaS delivery model	No software installation for the end user; the same document opens identically on any device with a browser
Global CDN / Load Balancer	Content Delivery Network (IaaS/PaaS networking)	Routes each user to the nearest edge location, which is what keeps real-time editing feeling instant worldwide
Sync Web API	Real-time Web API layer	Pushes every keystroke/change to all connected clients so documents stay in sync without manual refresh
Document storage	Distributed cloud storage with versioning	Automatically keeps prior versions, so edits can be rolled back and the system tolerates node failures
Identity & Access Management	IAM / encryption (security-as-a-service)	Enforces who can view/edit each document and protects data at rest and in transit without custom-built security code

2.4 Analysis and Trade-offs

Real-time synchronisation across many concurrent editors is the hardest part of this system: it requires conflict resolution logic (e.g., operational transforms or CRDTs) so that two people typing in the same paragraph do not overwrite each other, which adds engineering complexity compared to a simple "save and overwrite" file model. Using a global CDN and load balancer improves latency for geographically spread teams but increases infrastructure cost and requires careful cache-invalidation so edits are never served stale. Storing every version of every document gives strong auditability and easy rollback, at the cost of higher storage consumption over time — a trade-off that is acceptable for an enterprise tool where losing work is far more costly than paying for extra storage.

2.5 Solution Design Summary

Clients connect purely through a browser to a global CDN/load balancer, which routes them to a collaboration service exposing a real-time sync API and an identity/permissions layer. Documents are persisted in distributed, version-controlled cloud storage with encryption at rest and in transit, giving the enterprise a scalable, secure, Google-Docs-style editing experience without operating any of the underlying servers itself.

Case Study 3: Online Food Ordering Platform

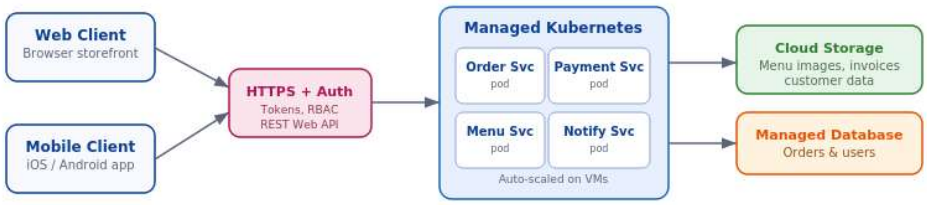
3.1 Understanding of the Case

A startup wants to let customers browse a menu and place an order from either a website or a mobile app, with the order flowing through to payment, kitchen, and delivery in real time. Because it is a startup, the platform must be cheap to run when traffic is low (few orders at 3 a.m.) but able to scale quickly during lunch/dinner rushes, all while protecting customer data, card-related tokens, and personal details.

Core requirements identified:

- A common backend reachable from both web and mobile clients via RESTful Web APIs
- Secure storage for menu images, invoices, and customer data
- Elastic, scalable infrastructure to absorb demand spikes without over-provisioning
- Authentication, HTTPS, and role-based access control to protect users and restaurant staff accounts

3.2 Proposed Cloud Architecture



Web and mobile clients call RESTful Web APIs over HTTPS, authenticated with tokens and role-based access control. Backend microservices run as auto-scaled pods on managed Kubernetes across virtual machines, while menu images, invoices, and customer/order records persist in cloud storage and a managed database.

Fig 3: Cloud architecture for the Online Food Ordering Platform

3.3 Application of Cloud Concepts

Requirement / Component	Cloud Concept Applied	Justification

Web & mobile clients	Thin clients over RESTful Web APIs	One shared API serves both platforms, so business logic is written once and reused everywhere
HTTPS + Auth layer	Token-based authentication, RBAC	Confirms caller identity on every request and limits staff/customer accounts to only the actions they need
Order, Payment, Menu, Notify services	Containerised microservices on managed Kubernetes (IaaS/PaaS)	Each service scales independently — Payment can scale differently from Menu browsing — and failures stay isolated
Virtual machines + Kubernetes	Infrastructure-as-a-Service with orchestration	Removes manual server provisioning; pods are scheduled automatically as demand rises and falls
Menu images, invoices, customer data	Cloud object storage + managed database	Durable, backed-up storage that scales with the number of restaurants and orders without capacity planning

3.4 Analysis and Trade-offs

Splitting the backend into independent microservices (order, payment, menu, notification) running on managed Kubernetes gives fine-grained, cost-efficient auto-scaling — the payment service can scale up during a flash sale while the menu service stays small — but it introduces the operational complexity of coordinating many small services instead of one monolith, along with the need for service discovery and monitoring. Using managed Kubernetes instead of self-managed VMs shifts patching and cluster upkeep to the cloud provider, which suits a small startup team, though it does mean less low-level control than a fully custom setup. Storing card-related data via tokenisation rather than raw card numbers adds a small amount of integration work up front but drastically reduces the platform's compliance burden and breach impact.

3.5 Solution Design Summary

Web and mobile clients call a shared RESTful API over HTTPS, authenticated with tokens and constrained by role-based access control. Requests are handled by auto-scaled microservices running on managed Kubernetes across virtual machines, and all persistent data — menu images, invoices, customer and order records — is kept in cloud storage and a managed database, giving the startup a platform that scales cheaply from its first order to its busiest rush hour.

Rubric Self-Check

The responses above were structured against the assessment rubric: each case study states the technical and business requirements drawn directly from the scenario (Understanding of Case), maps every component to a named cloud concept with a justification (Application of Concepts), discusses trade-offs behind each major design choice (Analysis), proposes a concrete, feasible architecture with a supporting diagram (Solution Design), and uses consistent headings, tables, and captioned figures throughout for readability (Clarity).